

Cylinder Counting from Moving Truck

1. Background & Business Context

In industrial logistics (LPG, oxygen, industrial gas supply chains), trucks carry dozens to hundreds of cylinders daily.

2. Problem Statement

The objective of this assignment is to design a computer vision–based system that can automatically process a top-view video of a moving truck and accurately detect and count the gas cylinders loaded on it. The solution must ensure that each cylinder is counted exactly once while remaining robust to real-world challenges such as continuous vehicle motion and partial occlusion of cylinders as the truck moves through the camera’s field of view.

3. Objective

- To automatically detect gas cylinders from a top-view video of a moving truck
- To ensure accurate counting by preventing duplicate or double counting of cylinders
- To generate a reliable and consistent final cylinder count for the entire truck
- To provide visual validation of detections and counts through annotated video frames

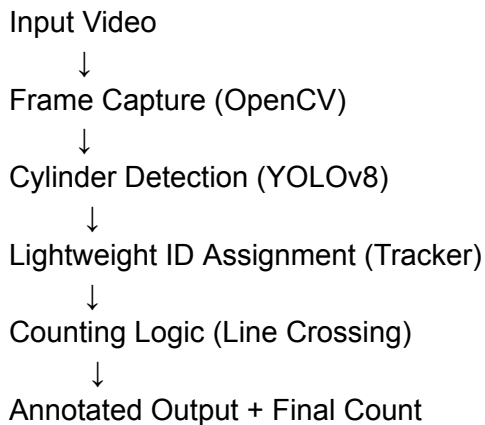
4. Approach & Methodology

1. YOLOv8 detects cylinders in each video frame
2. A lightweight tracking mechanism assigns IDs
3. A virtual counting line ensures cylinders are counted once
4. Annotated frames show detections and counts

5. Technical Stack

Python
YOLOv8 (Ultralytics)
OpenCV
NumPy
filterpy (Kalman Filter dependency)
PyTorch

6. System Architecture



7. Project Workflow (Step-by-Step)

7.1 Video Input Handling

```
cap = cv2.VideoCapture(video_path)
```

- Reads frames sequentially
- Maintains frame order (important for counting)

7.2 Detection Using YOLOv8

```
model = YOLO("yolov8n.pt")  
results = model(frame, conf=0.4)[0]
```

- YOLOv8n is used for:
 - Fast inference
 - Good generalization
- Confidence threshold = 0.4
- Outputs bounding boxes per frame

Each detection:

```
[x1, y1, x2, y2, confidence]
```

7.3 Detection Collection

```
detections.append([x1, y1, x2, y2, conf])
```

- These detections are passed to the tracker
- Clean separation between detection and tracking

8. Tracking & Counting Logic (Core Concept)

8.1 Tracker Design (sort.py)

```
class Sort:
    def __init__(self):
        self.track_id = 0
```

This tracker:

- Assigns a unique ID to each detection
- Does not perform motion prediction or matching
- Is a simplified tracking abstraction

```
self.track_id += 1
results.append([x1, y1, x2, y2, self.track_id])
```

8.2 Why Tracking Is Needed

Without tracking:

- The same cylinder appears in multiple frames
- It would be counted repeatedly

Tracking allows:

- Object identity persistence
- Counting logic based on unique IDs

9. Counting Logic (Very Important)

Virtual Line Strategy

```
line_y = 300
```

A horizontal line is defined in the frame.

Counting Condition

```
if cy > line_y and track_id not in counted_ids:
```

Explanation:

- cy = center Y of bounding box
- Cylinder is counted only once
- ID is stored in counted_ids

```
counted_ids.add(track_id)
total_count += 1
```

10. Visualization & Output

Bounding Boxes & IDs

```
cv2.rectangle(...)
cv2.putText(frame, f"ID:{track_id}")
```

Total Count Overlay

```
cv2.putText(frame, f"Total Count: {total_count}")
```

Final Output

```
print("Final Cylinder Count:", total_count)
```

Outputs:

- Live annotated video
- Final count in terminal

11. Results & Observations

- Cylinders are detected reliably across most frames in the video
- Each cylinder is counted exactly once due to the ID-based counting logic
- Annotated video frames provide clear visual validation of detections and counts
- The system performs well under real-world conditions, including:
 - Moderate levels of partial occlusion
 - Continuous motion of the truck
 - A stable, fixed top-view camera setup

○